

PreparedStatement and SQL Injection

Assumption for all code: MySQL with Connector/J on the classpath. Change the URL, username and password to match your setup.

Q1: AddSongToPlaylist

Table (for reference):

```
CREATE TABLE playlist_songs (
    song_id INT PRIMARY KEY,
    title   VARCHAR(100),
    artist  VARCHAR(100)
);

// AddSongToPlaylist.java
// Inserts a new song into the playlist_songs table using PreparedStatement.

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class AddSongToPlaylist {

    private static final String URL = "jdbc:mysql://localhost:3306/spotify_clone";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void addSong(int songId, String title, String artist) {
        String sql = "INSERT INTO playlist_songs (song_id, title, artist) VALUES (?, ?, ?)";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, songId);        // 1st ? -> song_id
            ps.setString(2, title);      // 2nd ? -> title
            ps.setString(3, artist);     // 3rd ? -> artist

            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("Song added successfully.");
            }
        } catch (SQLException e) {
            System.out.println("Error adding song: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        addSong(101, "Blinding Lights", "The Weeknd");
    }
}
```

Q2: FlipkartOrderManager.updateOrderStatus

```
// FlipkartOrderManager.java
// Updates the status of an order in the orders table using PreparedStatement.

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class FlipkartOrderManager {

    private static final String URL = "jdbc:mysql://localhost:3306/flipkart_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public void updateOrderStatus(int orderId, String newStatus) {
        String sql = "UPDATE orders SET status = ? WHERE order_id = ?";
    }
}
```

```

    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, newStatus);    // e.g. "Shipped", "Delivered"
        ps.setInt(2, orderId);

        int rows = ps.executeUpdate();
        if (rows > 0) {
            System.out.println("Order " + orderId + " updated to: " + newStatus);
        } else {
            System.out.println("No order found with ID " + orderId);
        }
    } catch (SQLException e) {
        System.out.println("Error updating order: " + e.getMessage());
    }
}

public static void main(String[] args) {
    new FlipkartOrderManager().updateOrderStatus(5001, "Shipped");
}
}

```

Q3: deleteCartItem

```

// CartManager.java
// Deletes a product from a user's cart using PreparedStatement.

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class CartManager {

    private static final String URL = "jdbc:mysql://localhost:3306/shop_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void deleteCartItem(int cartItemId) {
        String sql = "DELETE FROM cart_items WHERE cart_item_id = ?";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, cartItemId);

            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("Cart item " + cartItemId + " deleted.");
            } else {
                System.out.println("No cart item found with ID " + cartItemId);
            }
        } catch (SQLException e) {
            System.out.println("Error deleting cart item: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        deleteCartItem(12);
    }
}

```

Q4: Refactoring the buggy code

Buggy code (vulnerable):

```

String query = "INSERT INTO users (username, password) VALUES ('" + username + "', '" + password + "')";
Statement stmt = conn.createStatement();
stmt.executeUpdate(query);

```

Refactored code (safe):

```

String query = "INSERT INTO users (username, password) VALUES (?, ?)";

```

```
PreparedStatement pstmt = conn.prepareStatement(query);
pstmt.setString(1, username);
pstmt.setString(2, password);
pstmt.executeUpdate();
```

Explanation: In the buggy version, user input is joined directly into the SQL text, so an attacker can type SQL code (like ' '); DROP TABLE users; --) and change the query. With PreparedStatement, the SQL structure is sent to the database first with ? placeholders, and the user values are passed separately as plain data, so they can never be run as SQL commands.

Q5: Real-world SQL Injection scenarios

1. Paytm (payments and wallet app)

If the login or "check wallet balance" query is built by joining user input into the SQL text, an attacker could enter something like ' OR '1'='1 as the username and log in without knowing any password. They could then view other users' transaction history, wallet balances and linked bank details, or change the amount and receiver in a transaction record. This could lead to money theft, leaked financial data and loss of customer trust, plus legal penalties for the company.

2. Instagram (social media app)

If the search box, profile lookup or comment feature does not use safe queries, an attacker could inject SQL to read the entire users table. That could expose usernames, emails, phone numbers and password hashes of millions of people. They could also delete or edit posts and profiles, or take over accounts, including those of celebrities and businesses. The result would be a large privacy breach, account hijacking and reputation damage for the platform.

Prevention in both cases: use PreparedStatement (parameterized queries), validate input, give the database account only the permissions it needs, and store passwords hashed rather than as plain text.